

# ZEROHUNGER SYNC

Product Requirements Document (PRD) &  
Software Requirements Specification (SRS)

**Project Version:** 1.0.0 (MVP)

**Target Deployment:** Islamabad Pilot

**Classification:** Open-Source Community Platform

*An automated food rescue platform connecting surplus food providers with verified NGO responders via hyper-local, time-sensitive matching.*

# PART I: PRODUCT REQUIREMENTS DOCUMENT (PRD)

## 1. Executive Summary

---

Urban food insecurity and large-scale food waste are coexisting crises. In Islamabad, commercial food establishments routinely discard hundreds of kilograms of edible surplus food nightly. The root cause is a logistical and communication breakdown between those holding surplus and those requiring it. **ZeroHunger Sync** is an open-source, automated platform designed to close this gap.

This document outlines the Minimum Viable Product (MVP) aimed at deploying a hyper-local matching engine. The MVP restricts scope to essential functions to ensure a fast, reliable launch: strict user verification (Admin), simple donation logging (Providers), and real-time claim functionality (Consumers). By removing manual coordination overhead, ZeroHunger Sync facilitates frictionless handoffs of surplus food.

## 2. Target Audience & Personas

---

The platform serves three distinct user roles, ensuring secure and verified transactions:

- **Providers (Donors):** Restaurants, wedding halls, buffet operators, or individuals with surplus food. Their primary need is a fast, frictionless way to offload food before it spoils, without dealing with logistical headaches.
- **Consumers (NGOs/Community Coordinators):** Vetted charities and volunteer networks (e.g., the pilot partner charity in Islamabad). They need real-time alerts about available food nearby so they can dispatch volunteers efficiently.
- **Administrators:** Platform managers responsible for maintaining the integrity of the network. They review and approve registration applications to ensure food safety and operational reliability.

## 3. MVP Scope & Objectives

---

### Core Objective

To facilitate the immediate, unmediated transfer of surplus food data from a Provider to a localized network of Consumers, enabling a "first-to-claim" pickup model.

To keep the scope minimal while maximizing utility, the MVP will focus strictly on the following:

- **Role-based Authentication & Onboarding:** All users must register and be verified before participating.
- **Admin Dashboard:** A centralized view for admins to process pending user registrations and monitor basic platform activity.
- **Surplus Listing Engine:** A form for Providers to input food details (quantity, type, time window).
- **Alert & Claim System:** Automated notifications to nearby Consumers, with a concurrency-safe "Claim" mechanism.

**Out of Scope for MVP:** Real-time GPS delivery tracking, integrated messaging/chat, complex routing algorithms, mobile apps (MVP will be a responsive web app), and automated food quality inspections.

## 4. User Stories

---

### 4.1 Administration

- As an Admin, I want to view a dashboard of pending Provider and Consumer registrations so that I can vet them before granting platform access.
- As an Admin, I want to approve or reject accounts so that bad actors or unverified entities cannot access the network.

### 4.2 Providers (Donors)

- As a Provider, I want to register for an account by providing my establishment details, location, and contact info so I can be vetted.
- As a Provider, I want to quickly log a surplus food listing (specifying type, approx. servings/kg, and available pickup time window) so that local NGOs are notified.
- As a Provider, I want to see when my listing has been claimed and view the details of the NGO coming to pick it up.

### 4.3 Consumers (NGOs)

- As an NGO Coordinator, I want to register my organization and define our operating radius so we only receive relevant alerts.
- As an NGO Coordinator, I want to receive notifications (dashboard/email) when surplus food is posted within our radius.
- As an NGO Coordinator, I want to click a "Claim" button on an active listing to lock it in for my organization, preventing double-bookings.

- As an NGO Coordinator, I want to see the exact address and pickup instructions only after successfully claiming the listing.

## 5. Key Features

---

| Feature                      | Description                                                                                                                 | Priority |
|------------------------------|-----------------------------------------------------------------------------------------------------------------------------|----------|
| <b>Reg &amp; Auth Module</b> | Sign-up forms tailored to role (Provider vs. Consumer). Secure login via JWT/Session. Accounts default to "Pending" state.  | High     |
| <b>Admin Approval Portal</b> | A secure route displaying tabular data of pending accounts. Actions: Approve, Reject, Suspend.                              | High     |
| <b>Listing Creator</b>       | A minimalist form for Providers: Food Type (Perishable/Non-Perishable), Servings (Number), Expiry/Pickup Window (Datetime). | High     |
| <b>Radius Matching</b>       | Backend logic to calculate distance between the Provider and registered Consumers.                                          | High     |
| <b>Claim Engine</b>          | Concurrency-handled endpoint where the first Consumer to click "Claim" secures the listing. Status changes to 'Claimed'.    | High     |

# PART II: SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

## 1. Introduction

---

This section defines the technical specifications, architectural decisions, and functional requirements for the ZeroHunger Sync MVP. Designed for portability, the system leverages open-source, lightweight web technologies to allow easy deployment by any community globally.

## 2. System Architecture

---

The system will follow a standard Client-Server architecture utilizing a monolithic backend for simplicity and ease of deployment, which is crucial for the open-source mandate.

**Client: Responsive Web App (React.js or Vanilla JS/HTML/CSS)**

↓ ↑ REST API (JSON)

**Server: Node.js / Express.js or Python / Django**

↓ ↑ ORM / Queries

**Database: PostgreSQL (Relational integrity, PostGIS for geospatial)**

## 3. Functional Requirements

---

### 3.1 Authentication & Authorization (REQ-AUTH)

- **REQ-AUTH-01:** The system shall provide secure local authentication (Email/Password) using modern hashing (e.g., bcrypt).
- **REQ-AUTH-02:** Registration shall require a Role selection (Provider, Consumer). Admin accounts must be created via database seeding or CLI, not public registration.
- **REQ-AUTH-03:** Upon registration, user accounts shall have a `status` of `PENDING`. Pending users cannot create or claim listings.

- **REQ-AUTH-04:** The system shall utilize Role-Based Access Control (RBAC) to restrict endpoints (e.g., only Consumers can access `/api/claim`).

### 3.2 Admin Dashboard (REQ-ADM)

- **REQ-ADM-01:** The system shall provide an admin-only endpoint returning all users with `status='PENDING'`.
- **REQ-ADM-02:** The Admin interface shall allow the admin to update user status to `'APPROVED'` or `'REJECTED'`.
- **REQ-ADM-03:** The Admin dashboard shall display high-level metrics: Total Active Listings, Total Claimed Listings, Total Registered Entities.

### 3.3 Surplus Management & Matching (REQ-SUR)

- **REQ-SUR-01:** Providers shall be able to submit a `POST` request to create a listing. Required fields: `food_description`, `estimated_servings`, `pickup_start_time`, `pickup_end_time`.
- **REQ-SUR-02:** Upon listing creation, the backend shall perform a geospatial query to identify all `'APPROVED'` Consumers within a predefined radius (e.g., 10 km).
- **REQ-SUR-03:** The system shall generate notification records (or send emails) to the matched Consumers.
- **REQ-SUR-04:** Listings shall default to a `status` of `'AVAILABLE'`. If `pickup_end_time` passes without a claim, the status shall automatically update to `'EXPIRED'`.

### 3.4 Claiming Mechanism (REQ-CLM)

- **REQ-CLM-01:** Consumers shall view a feed of `'AVAILABLE'` listings within their radius.
- **REQ-CLM-02:** To claim, a Consumer issues a `POST` to `/api/listings/{id}/claim`.
- **REQ-CLM-03:** *Concurrency Control:* The system **MUST** implement database-level locking (e.g., optimistic concurrency or `'SELECT ... FOR UPDATE'`) to ensure only the first request successfully claims the listing. Subsequent concurrent requests must receive a `'409 Conflict'` or `'400 Bad Request'` indicating the listing is already claimed.
- **REQ-CLM-04:** Upon successful claim, the listing `status` becomes `'CLAIMED'`, the `consumer_id` is recorded, and the Provider is notified.
- **REQ-CLM-05:** Only the Consumer who successfully claimed the listing shall be authorized to view the Provider's exact street address and direct contact number.

## 4. Data Models (Core Schema)

Below is the relational structure necessary to support the MVP. Note: Geospatial data can be stored as simple latitude/longitude floats for the MVP to avoid complex database dependencies, though PostGIS is recommended for scaling.

| Table           | Key Fields                                                                                                                                         | Description                                                 |
|-----------------|----------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------|
| <b>Users</b>    | id (PK), email, password_hash, role (Enum), status (Enum: PENDING, APPROVED), name, phone, latitude, longitude, address                            | Stores authentication and profile data for all roles.       |
| <b>Listings</b> | id (PK), provider_id (FK), description, servings, pickup_start, pickup_end, status (Enum: AVAILABLE, CLAIMED, EXPIRED), consumer_id (FK, nullable) | The core transactional record representing a food donation. |

## 5. Non-Functional Requirements

- **Security:** All API communication must be strictly over HTTPS. API endpoints must validate user authorization tokens. Input data must be sanitized to prevent SQL injection and XSS.
- **Performance:** The listing claim endpoint (`/claim``) must respond within 500ms to provide a snappy experience and resolve race conditions quickly.
- **Usability:** The frontend must be highly responsive. Providers (often busy restaurant staff) must be able to complete a listing submission in under 60 seconds.
- **Portability & Open Source:** The codebase must include Docker and ``docker-compose`` configuration to allow a single-command spin-up of the application, database, and backend, fulfilling the "deploy anywhere" mandate.

### Deployment Strategy - Islamabad Pilot

The initial deployment will be hosted on a low-cost cloud provider (e.g., DigitalOcean or AWS Lightsail) running a single containerized instance. The pilot phase will onboard 10-15 restaurants and 1 partner charity. System logs and user feedback from this phase will drive the V2 roadmap.